

PROJETO MARIDO DE ALUGUEL

DOCUMENTO DE ARQUITETURA E REQUISITOS (AEP 1º BIMESTRE)

Instituição: Centro Universitário de Maringá, Unicesumar (Maringá, PR)

Curso: Engenharia de Software

Objetivo de Desenvolvimento Sustentável: ODS 8, Trabalho Decente e Crescimento Econômico

Integrante	RA
Eduardo Gomes Michelin	25068570-2
Pedro Henrique de Souza Castilho	25000689-2
Pietro Borim Berbert	25177983-2

1. DESCOBERTA E CONCEPÇÃO (O PROBLEMA E A ODS)

Douglas tem 41 anos e trabalha há mais de uma década com pequenos reparos residenciais em Maringá, fazendo coisas como trocar a resistência de um chuveiro, montar móveis, consertar vazamentos ou instalar prateleiras. Sua agenda depende quase inteiramente da indicação boca a boca e de mensagens em grupos de bairro no WhatsApp. Quando a indicação não chega, o dia fica ocioso e o mês fecha abaixo do que ele precisava. Do outro lado do problema está Camila, que acabou de se mudar para um apartamento e precisa resolver três reparos pequenos. Ela não conhece ninguém na cidade, pede indicações na portaria, recebe dois contatos sem qualquer histórico verificável e acaba negociando o preço sem nenhum parâmetro de comparação.

O que existe hoje entre Douglas e Camila é um mercado informal, opaco e de baixa confiança. O prestador autônomo competente não tem vitrine, e o contratante não tem como avaliar procedência, comparar valores ou prever prazo. O resultado é conhecido: preços dispersos para o mesmo serviço, contratações desfeitas por insegurança, retrabalho e uma parcela relevante de profissionais qualificados operando bem abaixo da própria capacidade produtiva.

Este projeto se alinha diretamente à **ODS 8 da ONU (Trabalho Decente e Crescimento Econômico)**. Ao dar visibilidade digital, reputação verificável e previsibilidade de demanda ao trabalhador autônomo, a plataforma atua sobre a informalidade não pela via da fiscalização, mas pela via do acesso ao mercado: o prestador passa a ter um histórico próprio, portátil e auditável do trabalho que realiza, o que amplia sua renda e reduz a assimetria de informação que hoje penaliza tanto quem contrata quanto quem executa. De forma complementar, a solução também dialoga com a ODS 11 (Cidades e Comunidades Sustentáveis), ao priorizar a contratação de profissionais próximos e fortalecer a economia de serviços do próprio bairro.

Resumo do Discovery (Matriz CSD e Jornada)

Durante a fase de descoberta, o mapeamento da Jornada do Usuário mostrou que o momento crítico não é a busca, e sim a decisão comparativa: o contratante só avança quando consegue colocar dois ou três profissionais lado a lado e pesar, ao mesmo tempo, preço, avaliação e distância. Enquanto essa comparação acontece de cabeça, ou espalhada entre abas e conversas diferentes, a contratação trava. Por isso, a tela de comparação foi tratada como núcleo do produto, e não como funcionalidade acessória.

A Matriz CSD (Certezas, Suposições e Dúvidas) revelou, por sua vez, uma premissa de negócio inegociável: a reputação precisa ser verificável. Uma nota que qualquer pessoa pode publicar sem ter contratado nada não gera confiança, apenas ruído. Ficou definido, portanto, que a avaliação é vinculada ao ciclo de vida da ordem de serviço: só é possível avaliar um prestador a partir de uma ordem efetivamente registrada e concluída na plataforma, uma única vez por ordem e sem edição posterior. Essa restrição atravessa o modelo de dados, as regras de negócio e a arquitetura da solução.

1.1 Partes interessadas (Stakeholders)

Douglas e Camila representam, na prática, os dois papéis centrais do sistema, o prestador e o cliente, mas a descoberta também identificou outras partes cujo interesse na solução precisa ser considerado formalmente na definição do escopo.

Stakeholder	Papel	Principal interesse	Problema / dor
Cliente (ex.: Camila)	Contrata serviços de reparo residencial pela plataforma	Encontrar um profissional confiável, comparando preço, nota e distância antes de contratar	Não tem parâmetro de comparação nem histórico verificável dos prestadores indicados
Prestador (ex.: Douglas)	Presta serviços autônomos de reparo residencial	Ampliar a carteira de clientes e ter uma renda mais previsível	Depende quase exclusivamente de indicação boca a boca, sem vitrine digital própria
Administrador da plataforma	Modera cadastros e acompanha o funcionamento geral do sistema	Garantir que ofertas e avaliações sigam as regras de negócio definidas	Risco de ofertas mal cadastradas ou avaliações fora do fluxo comprometerem a confiança na reputação exibida

Equipe de desenvolvimento (grupo AEP)	Projeta, implementa e evolui o sistema ao longo das sprints	Entregar um MVP funcional e coerente com os requisitos levantados na descoberta	Precisa equilibrar o prazo acadêmico do bimestre com a qualidade técnica do software entregue
---	---	---	---

2. LISTA DE REQUISITOS FUNCIONAIS (ESCOPO)

- **RF01.** O sistema deve permitir o cadastro e a autenticação de usuários, informando nome completo, e-mail, telefone, senha, rua, bairro, cidade e tipo de perfil, que pode ser cliente ou prestador. O e-mail deve ser único no sistema e cada perfil deve possuir permissões de acesso próprias.
- **RF02.** O sistema deve permitir que o prestador cadastre suas ofertas de serviço, informando nome do serviço, categoria, descrição, cidade e bairros atendidos, modelo de cobrança, que pode ser por hora ou por empreitada, e o valor base correspondente.
- **RF03.** O sistema deve permitir que o cliente busque ofertas de serviço por categoria, aplicando filtros de faixa de preço, nota mínima do prestador e região de atendimento, considerando cidade e bairro informados no cadastro. Os resultados podem ser ordenados por preço e por nota.
- **RF04.** O sistema deve permitir a comparação simultânea de até três ofertas de serviço escolhidas pelo cliente, mostrando em uma única tela o nome do prestador, o valor estimado, a nota média, a quantidade de serviços concluídos e a região de atendimento de cada oferta.
- **RF05.** O sistema deve permitir a abertura e o acompanhamento de ordens de serviço, registrando o cliente, o prestador, a oferta contratada, a data de abertura e o status atual. A ordem percorre os estados Aberta, Aceita, Em execução, Concluída e Cancelada. Cada alteração de status deve ser registrada em um histórico contendo o novo status, a data, a hora e o usuário responsável pela alteração.
- **RF06.** O sistema deve permitir que o cliente avalie o prestador após a conclusão da ordem de serviço, atribuindo uma nota de 1 a 5, um comentário e a data do registro. Cada ordem de serviço pode receber no máximo uma avaliação, e a nota média do prestador deve ser recalculada automaticamente a cada nova avaliação válida.

- **RF07.** O sistema deve permitir a geração de relatórios consolidados por prestador, reunindo a quantidade de serviços concluídos, o faturamento total do período e a evolução da nota média ao longo do tempo.

2.1 Requisitos não funcionais

- **RNF01 (Desempenho).** A listagem de resultados de busca com filtros aplicados deve ser devolvida em até 2 segundos, considerando uma base de até 5.000 prestadores cadastrados.
- **RNF02 (Segurança).** As senhas devem ser armazenadas em formato de hash com salt (BCrypt), a comunicação entre cliente e servidor deve ocorrer sobre HTTPS, e cada endpoint deve validar o perfil do usuário autenticado antes de executar a operação.
- **RNF03 (Usabilidade).** A interface deve ser responsiva a partir de 320 px de largura e funcionar como uma aplicação de página única (SPA), sem recarregamento completo entre as etapas de busca, comparação e contratação.
- **RNF04 (Integridade).** Cada avaliação deve estar vinculada a exatamente uma ordem de serviço concluída e permanecer imutável após o envio, o que garante a rastreabilidade da reputação exibida.

3. MATRIZ DE RASTREABILIDADE

A tabela a seguir mostra que os requisitos funcionais não foram definidos de forma arbitrária. Cada um responde a uma dor identificada na descoberta e se apoia em uma regra de negócio concreta, implementada por uma classe ou entidade específica do modelo.

Dor / problema	Requisito	Regra de negócio	Classe / entidade
Prestador sem vitrine digital, depende de indicação boca a boca	RF01, RF02	Cada prestador deve informar categoria, raio de atendimento e modelo de cobrança para cada oferta cadastrada	Prestador, OfertaServico
Cliente não consegue comparar preço, avaliação e distância entre profissionais	RF03, RF04	Os resultados de busca devem ser ordenáveis por preço, nota mínima e distância	OfertaServico, Prestador
Contratação informal, sem registro do que foi combinado nem do andamento	RF05	Toda contratação segue o ciclo Aberta, Aceita, Em execução, Concluída ou Cancelada	OrdemServico

Reputação não confiável, sem garantia de que quem avalia de fato contratou	RF06	Uma avaliação só pode ser registrada uma única vez, vinculada a uma ordem de serviço concluída	Avaliacao, OrdemServico
Prestador não tem visão consolidada do próprio desempenho	RF07	A nota média e o faturamento exibidos devem refletir apenas ordens efetivamente concluídas	Prestador, OrdemServico

4. O PLANEJAMENTO ÁGIL (CRONOGRAMA / BACKLOG)

A seguir, apresenta-se o fatiamento vertical das entregas, simulando o roadmap de desenvolvimento do 2º Bimestre. Cada sprint entrega uma fatia funcional completa, da interface ao banco de dados, de modo que exista software utilizável ao final de cada ciclo, e não apenas camadas técnicas isoladas.

Sprint / Data	Épico	User Story	Responsável
Sprint 1 05/09 a 19/09	Identidade e Acesso	COMO UM prestador autônomo EU QUERO criar minha conta e manter meu perfil profissional PARA QUE eu tenha uma vitrine própria e deixe de depender apenas de indicação.	Eduardo Michelon
Sprint 2 20/09 a 04/10	Catálogo e Busca	COMO UM cliente EU QUERO buscar profissionais por categoria e filtrar por preço, nota e distância PARA QUE eu encontre rapidamente quem atende o meu bairro.	Pedro Castilho
Sprint 3 05/10 a 19/10	Comparação e Ordem de Serviço	COMO UM cliente EU QUERO comparar até três profissionais lado a lado e abrir a ordem de serviço pelo próprio sistema PARA QUE eu contrate com segurança e sem negociar no escuro.	Pietro Berbert
Sprint 4 20/10 a 03/11	Reputação e Relatórios	COMO UM cliente EU QUERO avaliar o serviço após a conclusão e, COMO UM prestador, acompanhar meus indicadores PARA QUE a reputação exibida reflita apenas trabalhos realmente executados.	Eduardo, Pedro e Pietro

5. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

Para garantir um ecossistema robusto e tolerante a falhas, o backend da solução no 2º Bimestre será construído utilizando **Java com Spring Boot**. A escolha pela linguagem Java se justifica pela sua tipagem forte e pela sólida implementação dos pilares da Orientação a Objetos. O domínio do

problema exige regras de negócio rigorosas, como o cálculo de orçamento por diferentes modelos de cobrança, as transições controladas de estado da ordem de serviço, o recálculo de reputação e os perfis de acesso distintos, e o uso adequado de abstração, encapsulamento, herança e polimorfismo vai garantir um código testável, coeso e manutenível, em estrita aderência às exigências da disciplina de Engenharia de Software.

O Spring Boot organiza a aplicação em camadas bem delimitadas (Controller, Service e Repository), o que mantém as regras de negócio fora da camada de apresentação. Na prática, isso significa que a restrição central do produto, a de só permitir avaliação depois da conclusão, vive na camada de serviço, e não em uma validação de tela que poderia ser contornada por uma requisição direta à API.

No que diz respeito à persistência de dados e à estrutura de comunicação, o projeto vai adotar o banco de dados relacional **PostgreSQL**, orquestrado sob o padrão arquitetural de **API REST**. O PostgreSQL assegura a integridade referencial necessária para mapear as cardinalidades entre as entidades Usuário, Serviço, Ordem de Serviço e Avaliação, além de oferecer o tipo **NUMERIC** para valores monetários, o que evita os erros de arredondamento típicos de ponto flutuante em orçamentos e faturamento. Restrições declarativas de unicidade no banco reforçam, no último nível de defesa, a regra de uma única avaliação por ordem de serviço.

A abordagem via API REST permite o desacoplamento estratégico entre o frontend e o servidor. O frontend é uma aplicação de página única, desenvolvida em HTML, CSS e JavaScript, que consome os endpoints por meio de requisições assíncronas e mantém em memória o estado da comparação entre prestadores. Esse desacoplamento traz dois ganhos diretos ao projeto acadêmico: as equipes de frontend e backend podem evoluir em paralelo dentro da mesma sprint, e a mesma API poderá servir futuramente a um aplicativo móvel sem qualquer reescrita das regras de negócio.

6. ESTRUTURAÇÃO DO GITHUB E DIAGRAMAS

6.1 Link do repositório

Repositório Público GitHub: <https://github.com/souzapedro44/AEP-1entrega-1Bimestre.git>

6.2 Diagrama de classes (UML)

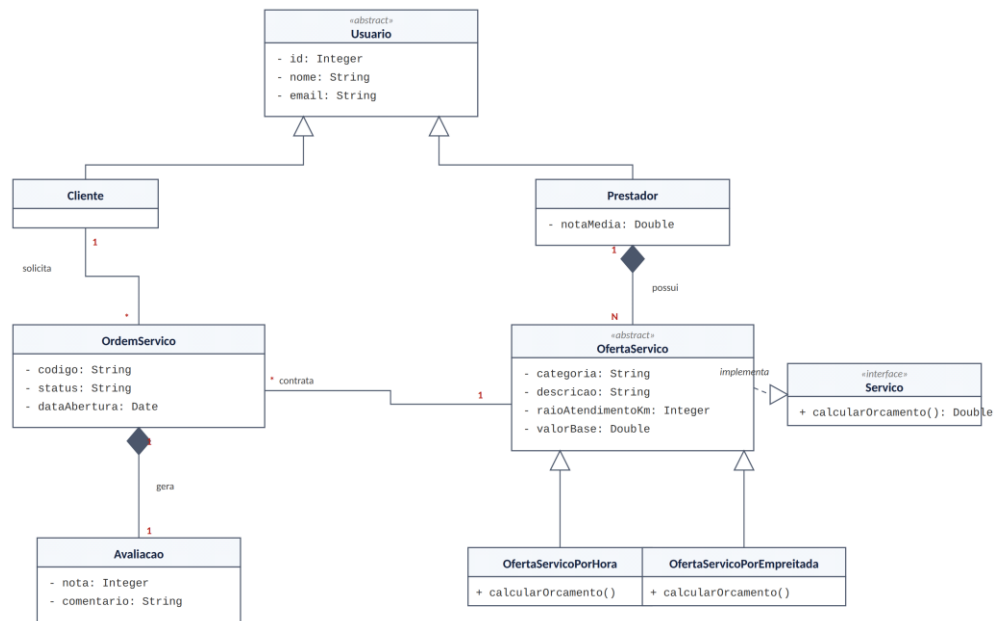


Figura 1. Diagrama de classes (UML) do sistema Marido de Aluguel.

- **Herança:** criação de uma superclasse genérica e abstrata chamada `Usuario`, com atributos comuns como `id`, `nome` e `email`. Dela derivam as subclasses específicas `Cliente` e `Prestador`, reaproveitando código e isolando responsabilidades: o atributo `notaMedia`, por exemplo, só faz sentido no prestador e não precisa poluir a superclasse.
- **Composição 1:N:** a classe `Prestador` é a classe pai, e `OfertaServico` é a classe filha. Cada oferta de serviço (categoria, descrição, raio de atendimento e modelo de cobrança) só existe porque um prestador específico a cadastrou. Ela não tem sentido nem pode ser acessada de forma independente do profissional que a criou, e se o cadastro do prestador for removido da plataforma, todas as suas ofertas deixam de existir junto. Como um mesmo prestador pode cadastrar diversas ofertas, por exemplo “instalação elétrica” e “montagem de móveis”, a multiplicidade do lado da parte é **N**, o que caracteriza uma composição 1:N e não apenas uma associação, já que o ciclo de vida da oferta está preso ao do prestador.
- **Polimorfismo:** a classe abstrata `OfertaServico` implementa a interface `Servico`, que define o método `calcularOrcamento()`. Esse método é sobrescrito (`@Override`) de formas distintas pelas subclasses `OfertaServicoPorHora`, que multiplica o valor base pela duração estimada do atendimento, e `OfertaServicoPorEmpreitada`, que devolve um valor fechado por escopo do serviço. A `OrdemServico` contrata sempre um objeto do tipo `OfertaServico` e invoca o mesmo método, mas a lógica de precificação executada muda dinamicamente conforme o tipo concreto da oferta contratada.

- **Associações 1:N:** um `Cliente` solicita múltiplas `OrdemServico`, e cada `OrdemServico` contrata exatamente uma `OfertaServico`, que por sua vez pode ser contratada por diversas ordens ao longo do tempo. Diferentemente da composição descrita acima, aqui a oferta continua existindo independentemente da ordem de serviço.
- **Encapsulamento:** todos os atributos são declarados como privados e acessados por métodos públicos. A `notaMedia` do prestador não possui método público de escrita: ela é recalculada internamente a cada nova avaliação válida, o que impede que a reputação seja alterada por fora das regras de negócio.

6.3 Diagrama de banco de dados (DER)

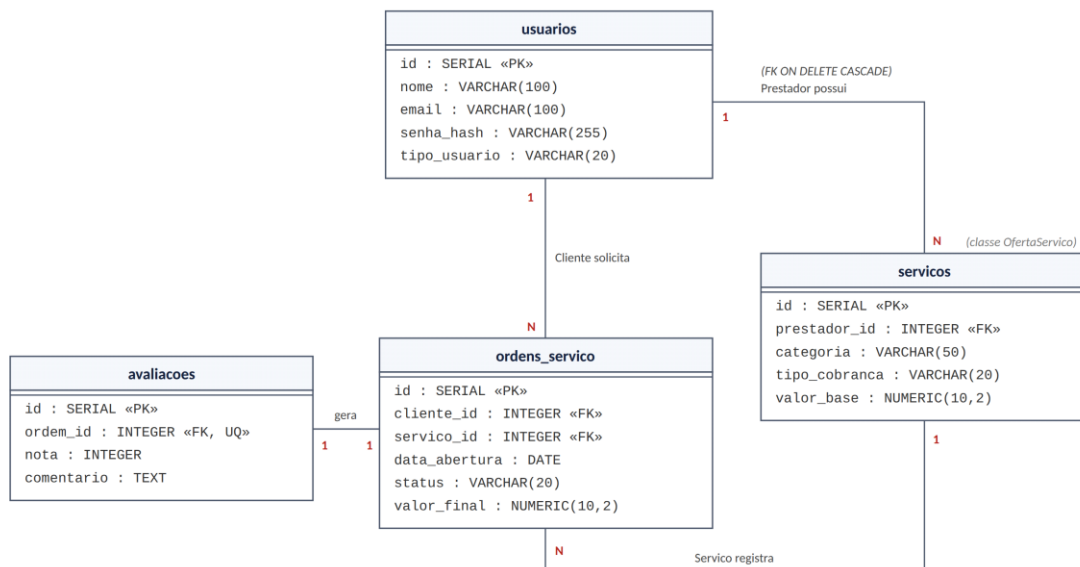


Figura 2. Diagrama entidade-relacionamento (DER) do banco PostgreSQL.

- **Tabela usuarios:** centraliza credenciais e dados pessoais, com uma coluna discriminadora (`tipo_usuario`) para distinguir clientes de prestadores. A senha nunca é armazenada em texto puro, apenas o seu hash.
- **Tabela servicos:** é a materialização relacional da classe `OfertaServico` do diagrama de classes. A chave estrangeira `prestador_id` referencia o prestador dono da oferta e é configurada com `ON DELETE CASCADE`: ao excluir um prestador, todas as suas linhas em `servicos` são removidas junto, reproduzindo no banco relacional a mesma composição 1:N que existe entre `Prestador` e `OfertaServico` no modelo de classes. A coluna `tipo_cobranca` é a contrapartida relacional do polimorfismo do diagrama: é ela que determina qual subclasse, `OfertaServicoPorHora` ou

OfertaServicoPorEmpreitada, será instanciada pela aplicação e qual implementação de calcularOrcamento() será executada.

- **Tabela** `ordens_servico`: materializa a contratação e resolve o relacionamento entre clientes e serviços, armazenando `cliente_id` e `servico_id`, além das colunas de rastreo `data_abertura`, `status` (Aberta, Aceita, Em execução, Concluída, Cancelada) e `valor_final`.
- **Tabela** `avaliacoes`: guarda a reputação verificável do prestador. A chave estrangeira `ordem_id` possui restrição de unicidade (`UNIQUE`), o que garante, no nível do próprio banco de dados, que cada ordem de serviço concluída produza no máximo uma avaliação, a tradução técnica da premissa levantada na Matriz CSD.

7. REFERÊNCIAS

- ORGANIZAÇÃO DAS NAÇÕES UNIDAS. *Objetivo de Desenvolvimento Sustentável 8: Trabalho decente e crescimento econômico*. Nações Unidas Brasil.
- SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019.
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.
- SPRING. *Spring Boot Reference Documentation*. VMware Tanzu.
- THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*.